

RAPPORT DE PROJET

Campus Service Booking Platform

Documentation Technique Complète

Domaine

Développement Web + API

Stack

React · FastAPI – Tailwindcss
· SQLite3

Année

2026

Table des Matières

01	Présentation du Projet Contexte, problème traité, objectifs et périmètre de la plateforme.
03	Instructions d'Installation & Lancement Guide pas-à-pas pour installer et démarrer le projet localement.
05	Captures d'Écran de l'Application Visuels des pages principales : landing, login, dashboard, réservations.
07	Difficultés Rencontrées & Solutions Problèmes techniques réels et approches adoptées pour les résoudre.

02	Outils & Technologies Utilisés Outils de développement, technologies frontend, backend et base de données.
04	Fonctionnalités Réalisées Authentification, gestion des services, réservations, dashboard et API REST.
06	Documentation des Routes API & Données de Test Endpoints REST documentés avec captures Postman et données de test.
08	Compétences Acquises & Améliorations Futures Bilan d'apprentissage et pistes d'évolution identifiées.

CHAPITRE 1

Présentation du Projet

Contexte & Problème Traité

Sur un campus universitaire, les étudiants doivent souvent effectuer des démarches administratives ou réserver des salles et services de manière informelle, sans outil centralisé. Cela engendre des pertes de temps, des demandes non tracées et une expérience utilisateur dégradée.

Campus Service Booking Platform est une application web full-stack permettant aux étudiants de réserver des services internes du campus de manière simple, rapide et traçable, avec une gestion des statuts par l'administration.

Objectifs

- Centraliser les demandes de services en un seul endroit
- Permettre aux étudiants de suivre l'état de leurs réservations
- Fournir une API REST robuste et normalisée
- Offrir une interface moderne et responsive

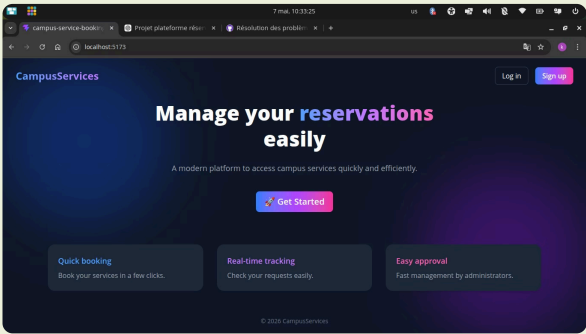


Figure 1 — Landing Page de la plateforme



Côté Étudiant

Créer un compte, se connecter, consulter les services disponibles et soumettre des réservations avec date et créneau.



Côté Administration

Visualiser toutes les réservations entrantes et mettre à jour leur statut (acceptée, rejetée).



API REST

Un backend FastAPI expose des endpoints standardisés pour gérer utilisateurs, services et réservations.

Outils & Technologies Utilisés

Outils de Développement

Outil	Version / Type	Rôle dans le projet
Visual Studio Code	Éditeur de code	Développement, organisation et débogage de l'application
Node.js	Runtime JavaScript	Gestion des dépendances et exécution des outils frontend (React)
Python	3.10	Développement de la logique backend et des API
Git	Gestion de versions	Suivi des modifications et gestion de l'historique du projet
GitHub	Hébergement de code	Stockage et partage du projet
Postman	Test d'API	Test et validation des endpoints backend

Frontend

- React**
Interface dynamique basée sur des composants
- React Router**
Gestion des pages et routes côté client
- Axios**
Requêtes HTTP vers le backend
- Tailwind CSS**
Interface moderne et responsive

Backend

- FastAPI**
Framework web Python pour API REST rapides
- Pydantic**
Validation et typage strict des données
- Uvicorn**
Serveur ASGI pour FastAPI
- bcrypt**
Hachage sécurisé des mots de passe

Base de Données

- SQLite3**
Base de données relationnelle légère intégrée à Python
- Schéma relationnel**
Tables : Utilisateurs, Services, Réservations
- Format ISO**
Dates stockées au format ISO pour compatibilité Python-SQLite

Instructions d'Installation & Lancement

Étape 1 — Installation des Outils de Développement

Outil	Lien / Commande
Visual Studio Code	https://code.visualstudio.com/
Node.js	https://nodejs.org/
Python 3.10	https://www.python.org/
Git	https://git-scm.com/
GitHub	https://github.com/
Postman	https://www.postman.com/

Étape 2 — Installation des Dépendances du Projet

Frontend

```
# Installer React Router
npm install react-router-dom

# Installer Axios
npm install axios

# Tailwind CSS
# Voir : https://tailwindcss.com/
```

Backend

```
# Installer FastAPI et Uvicorn
pip install fastapi uvicorn

# SQLite3 est intégré à Python
# Docs : https://www.sqlite.org/
```

Étape 3 — Lancer le Projet

1

Lancer le Backend (API)

Aller dans le dossier `app/` puis exécuter :

```
uvicorn backend.main:app --reload
```

2

Lancer le Frontend

Aller dans `app/frontend/campus-service-booking-platform/` puis exécuter :

```
npm run dev
```

❗ SQLite3 est directement intégré à Python — aucune installation séparée n'est nécessaire. La fonction `create_table()` est appelée automatiquement au démarrage de l'application pour initialiser les tables.

Fonctionnalités Réalisées

1

Authentification Utilisateur

- Inscription sécurisée avec hachage de mot de passe (bcrypt)
- Formulaire de connexion avec validation des identifiants
- Gestion de session avec localStorage
- Routes protégées — redirection automatique si non authentifié

2

Gestion des Services

- Affichage dynamique de tous les services disponibles
- Récupération via API REST (GET /services)
- Détails de chaque service (nom, description, statut)
- Sélection de service dans le formulaire de réservation

3

Système de Réservation

- Création avec sélection de service, date et créneau horaire
- Validation des données (date minimale, cohérence des horaires)
- Stockage en base SQLite
- Statuts : pending, accepted, rejected

4

Dashboard Utilisateur

- Réservations filtrées par utilisateur connecté
- Statistiques (en attente, acceptées, rejetées)
- Pagination (6 réservations par page)
- Recherche, filtrage et option d'annulation

5

Interface Utilisateur Moderne

- Design responsive avec Tailwind CSS
- Navigation avec Sidebar et Topbar
- Modal pour ajouter une réservation
- Toast notifications + support mobile/desktop

6

API REST Normalisée

- Routes conformes aux standards (GET / POST / PUT / DELETE)
- Endpoints clairs : /services, /bookings
- Gestion des erreurs HTTP appropriées
- Endpoint de profil utilisateur

Captures d'Écran de l'Application

Pages d'Authentification

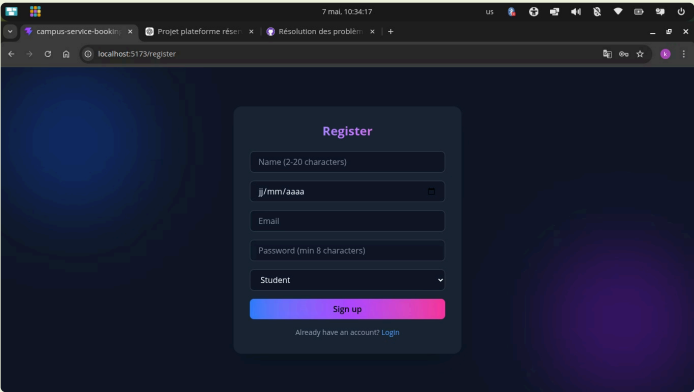


Figure 2 – Formulaire d'inscription (Register)

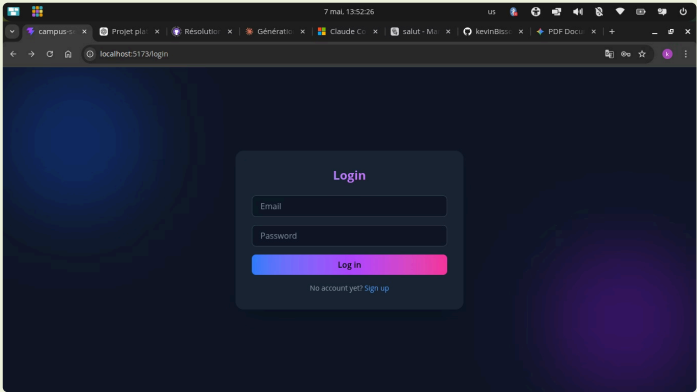


Figure 3 – Formulaire de connexion (Login)

Dashboard & Réservations

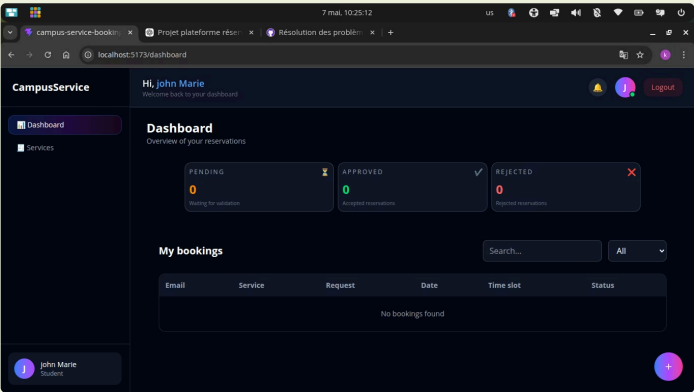


Figure 4 – Dashboard principal avec statistiques

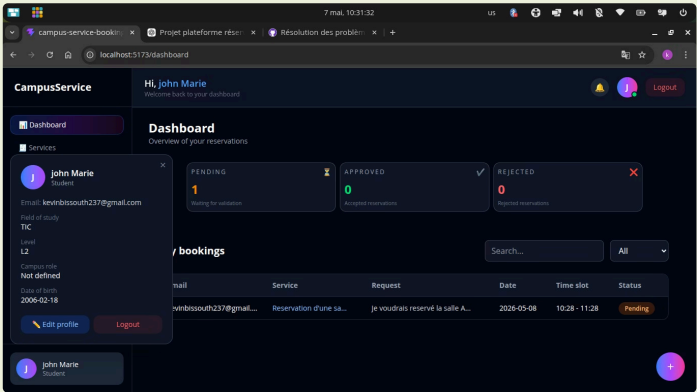


Figure 5 – Dashboard avec réservation active (profil)

Gestion des Services & Réservations

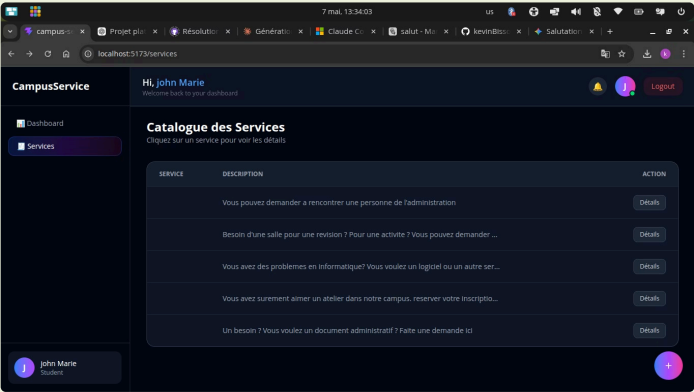


Figure 6 – Catalogue des Services disponibles

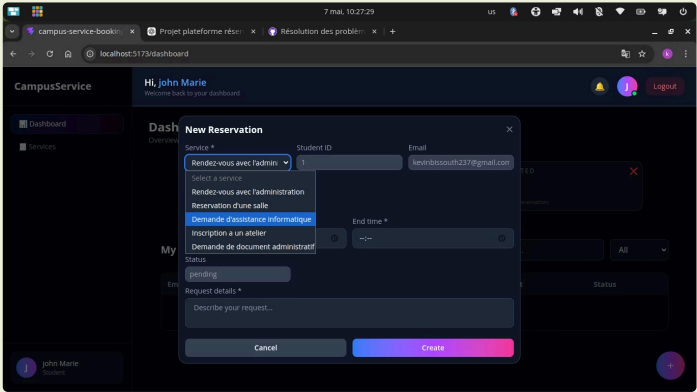
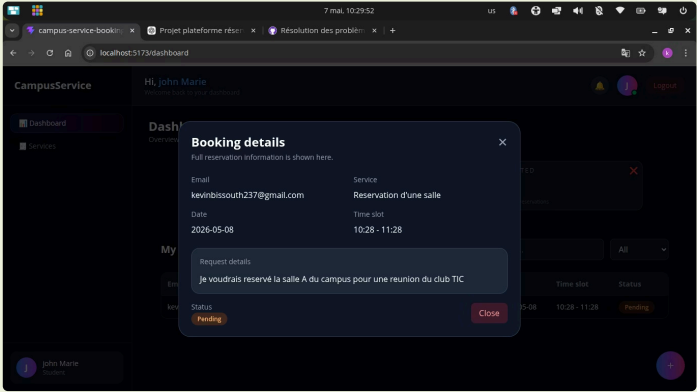
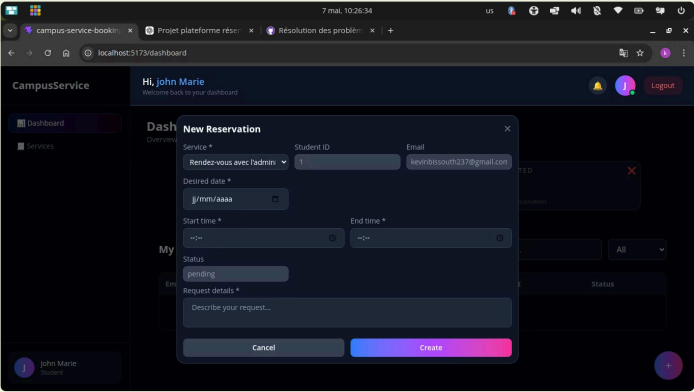


Figure 7 – Création de réservation : sélection du service



Documentation des Routes API & Données de Test

Routes API – Utilisateurs

Méthode	Endpoint	Description & Paramètres
POST	/add_user/	Frépiistrer un utilisateur. Champs : nom, date_de_naissance (YYYY-MM-DD), email, role (1=étudiant, 0=admin), mot_de_passe (min 8 car.)
GET	/get_all_users/	Récupérer tous les utilisateurs
GET	/get_user_by_id/{id}	Récupérer un utilisateur par son identifiant
PUT	/update_user/	Modifier un utilisateur (mêmes champs que POST)
DELETE	/delete_user/{id}	Supprimer un utilisateur par son identifiant
POST	/login/	Connecter un utilisateur. Champs : email, mot_de_passe

Routes API – Services

Méthode	Endpoint	Description & Paramètres
POST	/add_service/	Créer un service. Champs : nom, description, est_actif (1=disponible, 0=indisponible)
GET	/services	Récupérer tous les services
GET	/services/{id}	Récupérer un service par son identifiant
PUT	/update_service/{id}	Modifier un service (mêmes champs que POST)
DELETE	/delete_service/{id}	Supprimer un service par son identifiant

Routes API – Réservations

Méthode	Endpoint	Description & Paramètres
POST	/bookings	Créer une réservation. Champs : etudiant (id), admin (id), email, service_choisis (id), demande, date_souhaitee, creneau_horaire, statut
GET	/bookings	Récupérer toutes les réservations
GET	/bookings/{id}	Récupérer une réservation par son identifiant
PUT	/update_reservation/{id}	Modifier une réservation (mêmes champs que POST)
DELETE	/bookings/{id}	Supprimer une réservation par son identifiant

Données de Test – Captures Postman

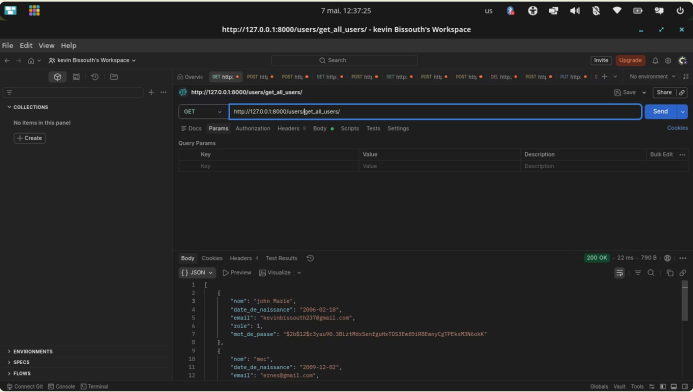


Figure 12 – Postman : GET /get_all_users/ – Réponse 200 OK avec liste des utilisateurs

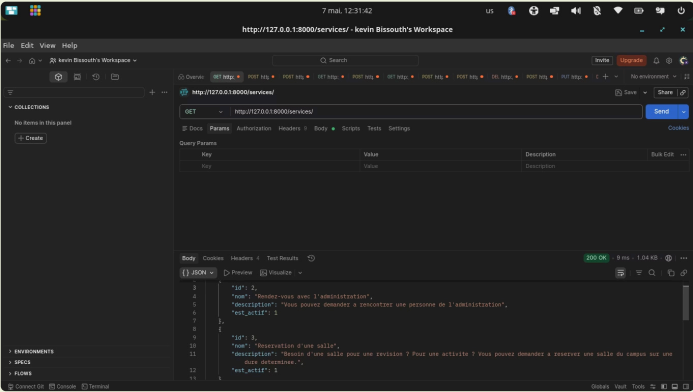


Figure 13 – Postman : GET /services/ – Liste complète des services

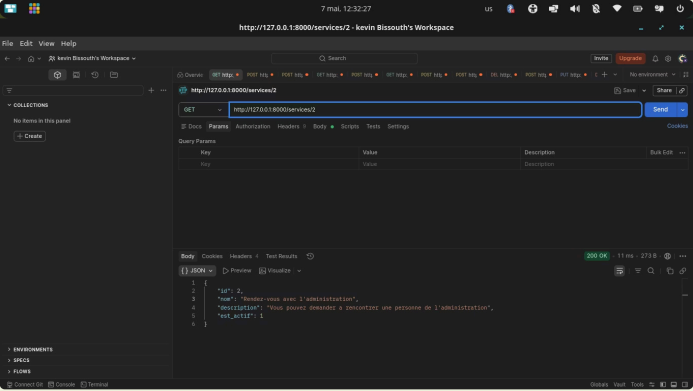


Figure 14 – Postman : GET /services/2 – Détail d'un service par ID

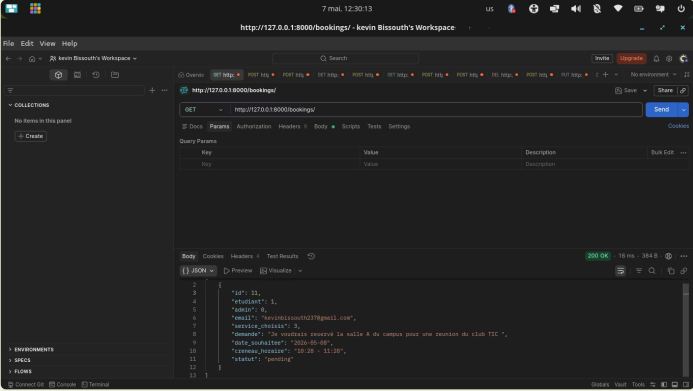


Figure 15 – Postman : GET /bookings – Toutes les réservations

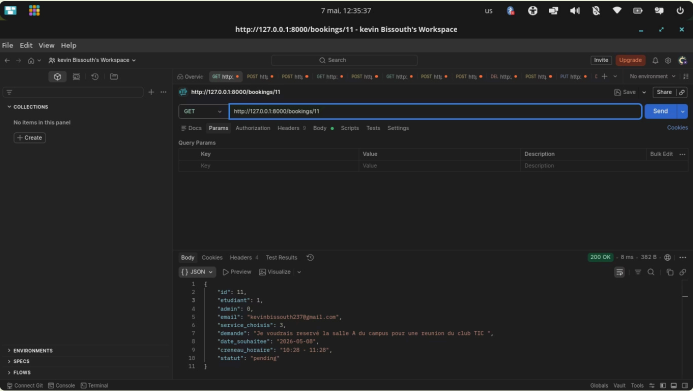


Figure 16 – Postman : GET /bookings/1 – Détail d'une réservation

 **Exemple de données de test – Réservation**
etudiant: 1
service_choisis: 3
email: kevinblissouth27@gmail.com
date_souhaitee: 2020-06-08
creneau_horaire: 10:00 - 11:00
statut: pending

CHAPITRE 7

Difficultés Rencontrées & Solutions



Gestion des Routes API

Problème : Routes non normalisées et incohérentes au départ (ex : `/get_all_services/`, `/add_reservation/`).

Solution : Refactorisation complète vers une architecture REST standard. Cela a également corrigé un bug de suppression de réservation qui appelait la mauvaise fonction.



Synchronisation Frontend-Backend

Problème : Les URLs d'API du frontend ne correspondaient pas toujours à celles du backend, causant des erreurs silencieuses.

Solution : Audit complet du codebase et mise à jour coordonnée de tous les appels Axios sur les 3 composants frontend concernés.



Validation des Données

Problème : Validation insuffisante des dates et horaires côté frontend permettant des réservations incohérentes.

Solution : Ajout de validations strictes (date minimale = aujourd'hui, vérification cohérence début/fin). Les erreurs sont détectées côté client avant envoi au backend.



Initialisation de la Base de Données

Problème : Les tables SQLite devaient être créées avant toute utilisation de l'application.

Solution : Implémentation d'une fonction `create_table()` appelée automatiquement au démarrage. Les dates sont gérées au format ISO pour compatibilité Python-SQLite.



Authentification & Sécurité

Problème : Les mots de passe ne pouvaient pas être stockés en clair. Par ailleurs, la gestion de session reposait uniquement sur `localStorage`.

Solution : Utilisation de `bcrypt` pour le hachage sécurisé. **Limitation actuelle :** pas de JWT tokens — amélioration identifiée pour une version future.



Réalisation du Frontend

Problème : React était une technologie nouvelle, difficile à appréhender pour construire les différentes pages de l'application.

Solution : Recherche active et parcours de la documentation officielle React (react.dev). **Limitation actuelle :** maîtrise partielle de la technologie.

CHAPITRE 8

Compétences Acquises & Améliorations Futures



Ce que l'Étudiant a Appris

- **Développement Full-Stack**

Architecture client-serveur, séparation frontend/backend, communication via API REST, gestion des états React (useState, useEffect, useContext).

- **Backend & APIs**

Design REST avec FastAPI, méthodes HTTP (GET/POST/PUT/DELETE), validation Pydantic, gestion des erreurs HTTP et status codes.

- **Base de Données**

SQL basique (SELECT/INSERT/UPDATE/DELETE), schémas relationnels, pagination, filtrage et optimisation des requêtes.

- **Sécurité & Versioning**

Hachage bcrypt, validation des inputs, Git/GitHub, commits signifiants et documentation du code.



Améliorations Possibles



Sécurité Renforcée

Implémenter JWT, authentification 2FA, refresh tokens et limite de tentatives de connexion.



Fonctionnalités Avancées

Notifications temps réel (WebSocket), export PDF, rappels email, calendrier interactif des disponibilités.



Performance & Infrastructure

Migration vers PostgreSQL, mise en cache Redis, pagination côté serveur, CDN pour les assets.



Tests & Qualité

Tests unitaires (Jest + pytest), tests E2E (Cypress), couverture de code à 80%+, documentation Swagger.



DevOps & Déploiement

Containerisation Docker, pipeline CI/CD (GitHub Actions), monitoring centralisé, déploiement cloud (AWS, Railway).



Ce projet full-stack a permis de mettre en pratique l'intégralité d'un cycle de développement web moderne : conception, développement, test et documentation — avec des technologies professionnelles utilisées en entreprise.